



Министерство науки и высшего образования Российской Федерации
Федеральное государственное
бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ

«Фундаментальные Науки»

КАФЕДРА

ФН-12 «Математическое моделирование»

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ НА ТЕМУ:

Обратная польская запись

Студент:

Подобедов В. В.

дата, подпись

Ф.И.О.

Преподаватель:

Волкова Л. Л.

дата, подпись

Ф.И.О.

Москва, 2023

Содержание

Введение	3
1 Аналитическая часть	4
2 Конструкторская часть	5
2.1 Проверка корректности инфиксной записи	5
2.2 Перевод в польскую запись	5
2.3 Вычисление значения функции в постфиксной записи	5
2.4 Написание стека	6
2.5 Функции программы	6
3 Технологическая часть	8
4 Исследовательская часть	20
Заключение	29
Список используемых источников	30

Введение

Цель лабораторной работы: разработать программу для вычисления значений функции с использованием обратной польской записи.

Для достижения поставленной цели требуется решить следующие **задачи**.

1. Описать две формы записи — инфиксную и постфиксную.
2. Разработать алгоритм проверки введенной инфиксной записи.
3. Реализовать алгоритм перевода из инфиксной записи в постфиксную форму записи.
4. Составить таблицу значений функции, при заданных границах и шаге аргумента.
5. Выполнить тестирование реализации разработанных алгоритмов.

1 Аналитическая часть

Инфиксная нотация — это форма записи математических и логических формул, в которой операторы записаны в инфиксном стиле между операндами, на которые они воздействуют. Иначе говоря, обычные арифметические выражения, используемые в повседневной практике и содержащие скобки.

Постфиксная нотация — форма записи математических и логических выражений, в которой операнды расположены перед знаками операций. Эта форма записи также называется Обратной польской записью.

Стек[1] — абстрактный тип данных, представляющий собой список элементов, организованных по принципу LIFO (англ. last in — first out, «последним пришёл — первым вышел»). Стек является очередью с одной стороной входа и выхода элементов.

2 Конструкторская часть

2.1 Проверка корректности инфиксной записи

Для проверки корректной записи скобок в выражении необходимо создать курсор и поэлементно обрабатывать данные, начиная с первого элемента до конечного. Если была найдена открывающая скобка “(”, то добавить единицу к целочисленной переменной. Иначе, если была найдена закрывающаяся скобка “)”, нужно вычесть единицу из переменной, которая отвечает за проверку количества закрывающихся и открывающихся скобок. Если переменная после всего обхода строки осталась равной нулю, значит баланс скобок соблюден, иначе вывести предупреждение, о том что баланс скобок не соблюден. Если переменная отрицательное значение, необходимо вывести ошибку и прекратить выполнение программы.

Для проверки корректности токенов, входящих в строку, необходимо также, как и с проверкой скобок запустить курсор, который будет идти по каждому элементу строки. Созданные функции будут проверять является ли элемент оператором, функцией, числом или переменной. Если все функции возвратили отрицательное значение, значит необходимо вывести ошибку, что строка содержит не допустимый токен. Также если нужно проверить, чтобы количество операторов было на единицу меньше, чем количество операндов. Иначе программа должна вывести ошибку. Если идут два оператора подряд (+, −, *, /), то вывести ошибку.

2.2 Перевод в польскую запись

Чтобы перевести выражение из инфиксной в постфиксную запись, необходимо создать курсор, который будет двигаться поэлементно от начала строки до конца. Также нужно реализовать стек, в который будут добавляться операторы. Если встретился элемент, который является числом или переменной, то добавить его в заранее созданную строку, которая будет хранить всю постфиксную запись и сместить курсор. Далее проверяется не встретилась ли открывающаяся скобка, если да, то добавить ее в стек и сдвинуть курсор. Если встретился символ — оператор (+, * и т.д.), проверить его приоритет относительно операторов в стеке. Если приоритет текущего оператора выше или равен приоритету оператора на вершине стека, поместить его в стек операторов. Если приоритет ниже, извлечь операторы из стека и добавить их в постфиксное выражение, пока не выполнится условие приоритета. Если встретился символ закрывающейся скобки, выполнить цикл извлечения операторов из стека и добавления их в постфиксное выражение до тех пор, пока не встретится открывающаяся скобка. После завершения обработки символов входной строки, извлечь все оставшиеся операторы из стека и добавить их в конец постфиксного выражения.

2.3 Вычисление значения функции в постфиксной записи

Для корректного вычисления значения функции, необходимо создать стек, который будет хранить операнды. Далее создается курсор, который будет идти по каждому элементу

в постфиксной строке и обрабатывать символы. Если встрилась цифра, то она добавляется в стек и курсор смещается. Если попался символ — арифметический оператор, то из стека извлекаются последние два операнда и выполняется арифметическая операция между этими элементами. Результат арифметической операции добавляется обратно в стек. Если встрелась функция, то из стека извлекается последний и находится синус или котангенс этого значения, результат помещается обратно в стек. После обработки всех символов входной постфиксной строки, извлекается последний элемент из стека, который и является результатом выражения. Если функция не определена в некоторой точке, то функция возвращает бесконечное значение и, дальше программа обрабатывает его, ставя в таблицу прочерк.

2.4 Написание стека

С помощью шаблонных классов “List”, который описывает саму структуру стека, и “Node”, который описывает блок стека, реализовать стек на ссылках. Требуется разработать следующие методы.

- Метод “push_back” — метод, который добавляет элемент в конец стека.
- Метод “pop_back” — метод, который удаляет последний элемент из стека.
- Метод “GetSize” — метод, который печатает количество элементов в стеке.
- Метод “check” — метод, который проверяет стек на пустоту.
- Метод “print” — метод, который выводит весь стек в консоль.
- Перегрузить оператор “[]” для вывода элемента стека на определённом месте.

2.5 Функции программы

Требуется разработать следующие функции.

- Функция “read_file” — функция, которая считывает инфиксную строку из файла в переменную.
- Функция “isNumber” — функция, которая проверяет является ли символ числом.
- Функция “isOperator” — функция, которая проверяет является ли элемент оператором.
- Функция “isFunction” — функция, которая проверяет является ли элемент функцией.
- Функция “valid_inf” — функция, которая проверяет является ли инфиксное выражение строкой.
- Функция “precedence” — функция, которая проверяет приоритет оператора.

- Функция “infixToPostfix” — функция, которая переводит строку из инфиксной записи в постфиксную.
- Функция “evaluatePostfix” — функция, которая высчитывает значение выражения, используя постфиксную форму записи.

К программе предъявлены следующие требования. Строка, которая содержит инфиксную форму записи считывается из файла, проверяется на корректность её записи и переводится в постфиксную запись. Далее пользователь вводит границы отрезка и шаг аргумента, на котором он хотел бы узнать значения функции. Выводится таблица со значениями функции.

3 Технологическая часть

Лабораторная работа была реализована на языке программирования C++. Кроме стандартной библиотеки `<iostream>`, которая предоставляет возможность пользоваться средствами ввода-вывода для стандартной консоли, была подключена библиотека `<fstream>`. Она включает в себя набор классов, методов и функций, которые предоставляют интерфейс для чтения/записи данных из/в файл. Также была подключена библиотека `<string>` для пользования строками. Был также написан стек на ссылках. Далее приведен листинг реализации стека (листинг 1).

Листинг 1 – Исходный код программы для реализации стека

```
#include <iostream>
#include <string>
#include <fstream>
#include <limits>
#include <locale>

using namespace std;

const double PI = 3.141592653589793;

template<typename T>
class List {
public:
    List() {
        size = 0;
        head = nullptr;
    }

    ~List() {}

    void push_back(T date) {
        if (head == nullptr) {
            head = new Node<T>(date);
        }
        else {
            Node<T> *current = this->head;
            while (current->pNext != nullptr) {
                current = current->pNext;
            }
            current->pNext = new Node<T>(date);
        }
    }
};
```



```

        size++;
    }

    int GetSize() {
        return size;
    }

    void pop_back() {
        Node<T> *previous = this->head;
        for (int i = 0; i < GetSize() - 2; i++) {
            previous = previous->pNext;
        }
        if (size > 1) {
            Node<T> *toDelete = previous->pNext;
            previous->pNext = toDelete->pNext;
            delete toDelete;
            size--;
        }
        else {
            Node<T> *toDelete = head;
            head = nullptr;
            delete toDelete;
            size--;
        }
    }

    void check() {
        if (size == 0) {
            cout << "Стек пустой" << endl;
        }
        else {
            cout << "В стеке хранятся данные " << endl;
        }
    }

    void print() {
        Node<T> *current = this->head;
        while (current != nullptr) {
            cout << current->date;
            current = current->pNext;
        }
    }
}

```

```

T& operator [] (const int index)
{
    int counter = 0;
    Node<T> *current = this->head;
    while (current != nullptr){
        if (counter == index){
            return current->date;
        }
        current = current->pNext;
        counter++;
    }
}

```

private:

```

template<typename T>
class Node {
public:
    Node * pNext;
    T date;

    Node(T date, Node * pNext = nullptr) {
        this->date = date;
        this->pNext = pNext;
    }
};

Node<T> *head;
int size;

};

void read_file(string &inf) {
    string filename = "C:\\Users\\Kononova\\Desktop\\lab_3_2_inf.txt"
        ;
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "Ошибка: Неудалось открыть файл  ." << endl;
        exit;
    }
    while (getline(file, inf)) {
        cout << "Прочитанная строка: " << inf << endl << endl;
    }
}

```

```

    }
    file.close();
}

bool isNumber(const string& s) {
    for (char c : s) {
        if (!isdigit(c) && c != '.' && c != '-') {
            return false;
        }
    }
    return true;
}

bool isOperator(string &c) {
    return (c == "+" || c == "-" || c == "*" || c == "/" || c == "^"
            || c == "(" || c == ")");
}

bool isFunction(string& token) {
    return (token == "sin" || token == "ctg");
}

bool valid_inf(string &inf) {
    int brackets = 0;
    for (int i = 0; i < inf.size(); i++) {
        if (i == 0 && inf[i] == '-') {
            inf.insert(0, "0 ");
        }
        if (i > 1 && inf[i] == '-' && inf[i - 2] == '(') {
            inf.insert(i, "0 ");
        }
        if (i > 1 && (inf[i] == '-' || inf[i] == '+' || inf[i] == '*' || inf[i] == '/')
            && (inf[i - 2] == '-' || inf[i - 2] == '+' || inf[i - 2] == '*' || inf[i - 2] == '/')) {
            cout << "Ошибка в валидности операторов " << endl << endl;
            return 0;
        }
        if (i > 1 && inf[i] == '-' && (inf[i - 2] == '/' || inf[i - 2] == '*')) {
            inf.insert(i + 4, ") ");
        }
    }
}

```

```

        inf.insert(i, "( 0 ");
    }
    if (inf[i] == '(') {
        brackets++;
    }
    else if (inf[i] == ')') {
        brackets--;
        if (brackets < 0) {
            cout << "Неправильно расставлены скобки " <<
                endl << endl;
            return false;
        }
    }
}

if (brackets != 0) {
    cout << "Неправильно расставлены скобки " << endl << endl;
    return false;
}

int operators = 0, operands = 0;
for (int i = 0; i < inf.size(); i++) {
    string check;
    while (i < inf.size() && inf[i] != ' ') {
        check += inf[i];
        i++;
    }
    if (isOperator(check)) {
        operators++;
        continue;
    }
    else if (isFunction(check)) {
        operators++;
        continue;
    }
    else if (isNumber(check)) {
        operands++;
        continue;
    }
    else if (check == "x") {
        operands++;
        continue;
    }
    else {

```

```

        cout << "Ошибка ввалидноститокенов " << endl << endl
        ;
        return 0;
    }
}

if (operands - 1 != operators) {
    cout << "Ошибка ввалидностиоператоров " << endl << endl;
    return 0;
}

return true;
}

int precedence(char op) {
    if (op == 's' || op == 'c')
        return 4;
    else if (op == '^')
        return 3;
    else if (op == '*' || op == '/')
        return 2;
    else if (op == '+' || op == '-')
        return 1;
    else
        return -1;
}

string infixToPostfix(const string& expression) {
    List<char> operators;
    string postfix;

    for (char c : expression) {
        if (c == ' ' || c == 'i' || c == 'n' || c == 't' || c ==
            'g') {
            continue;
        }
        if (c == '.') {
            postfix += c;
            continue;
        }
        if (c != 's' && c != 'c' && isalnum(c)) {
            postfix += c;
        }
        else if (c == '(') {

```

```

        operators.push_back(c);
    }
    else if (c == ')') {
        while (operators.GetSize() > 0 && operators[
            operators.GetSize() - 1] != '(') {
            postfix += ' ';
            postfix += operators[operators.GetSize()
                - 1];
            if (operators[operators.GetSize() - 1] ==
                's') {
                postfix += "in";
            }
            if (operators[operators.GetSize() - 1] ==
                'c') {
                postfix += "tg";
            }
            operators.pop_back();
        }
        if (operators.GetSize() > 0 && operators[
            operators.GetSize() - 1] == '(') {
            operators.pop_back();
        }
    }
    else {
        while (operators.GetSize() > 0 && precedence(c)
            <= precedence(operators[operators.GetSize() -
                1])) {
            postfix += ' ';
            postfix += operators[operators.GetSize()
                - 1];
            if (operators[operators.GetSize() - 1] ==
                's') {
                postfix += "in";
            }
            if (operators[operators.GetSize() - 1] ==
                'c') {
                postfix += "tg";
            }
            operators.pop_back();
        }
        if (c != 's' && c != 'c') {
            postfix += ' ';

```

```

        }
        operators.push_back(c);
    }
}

while (operators.GetSize() > 0) {
    postfix += ' ';
    postfix += operators[operators.GetSize() - 1];
    operators.pop_back();
}

return postfix;
}

double evaluatePostfix(const string& expression) {
    setlocale(0, "");
    List<double> operands;

    for(int i = 0; i < expression.size(); i++){
        if (expression[i] == ' ') {
            continue;
        }
        if (expression[i] == '-' && isdigit(expression[i + 1])) {
            int num = expression[i + 1] - '0';
            operands.push_back(-num);
            i = i + 2;
            continue;
        }
        if (isdigit(expression[i])) {
            if (expression[i + 1] == '.') {
                string numm;
                while (expression[i] != ' ') {
                    numm += expression[i];
                    i++;
                }
                locale::global(std::locale("en_US.UTF-8"));
                double num = std::stof(numm);
                operands.push_back(num);
                continue;
            }
            operands.push_back(expression[i] - '0');
        }
    }
}

```

```

}
else if (expression[i] == '+' || expression[i] == '-' ||
        expression[i] == '*' || expression[i] == '/' ||
        expression[i] == '^') {
    double operand2 = operands[operands.GetSize() -
        1];
    operands.pop_back();
    double operand1 = operands[operands.GetSize() -
        1];
    operands.pop_back();
    double result = 0.0;

    switch (expression[i]) {
    case '+':
        result = operand1 + operand2;
        break;
    case '-':
        result = operand1 - operand2;
        break;
    case '*':
        result = operand1 * operand2;
        break;
    case '/':
        if (operand2 == 0) {
            return std::numeric_limits<float>
                >::infinity();
        }
        result = operand1 / operand2;
        break;
    case '^':
        if (operand1 < 0 && (operand2 < 1 &&
            operand2 > 0)) {
            cout << "Square root of a
                negative number" << endl;
            return std::numeric_limits<float>
                >::infinity();
            break;
        }
        result = pow(operand1, operand2);
        break;
    }
}

```



```

        operands.push_back(result);
    }
    else if (expression[i] == 's') {
        double operand = operands[operands.GetSize() - 1];
        operands.pop_back();
        operands.push_back(sin(operand));
    }
    else if (expression[i] == 'c') {
        double operand = operands[operands.GetSize() - 1];
        operands.pop_back();
        if (PI / 2 == operand) {
            return std::numeric_limits<float>::infinity();
            break;
        }
        if (operand == 0) {
            return std::numeric_limits<float>::infinity();
            break;
        }
        operands.push_back(1.0 / tan(operand));
    }
}

return operands[operands.GetSize() - 1];
}

```

```

int main() {
    setlocale(0, "");
    string infix = "";
    read_file(infix);
    if (infix == "") {
        cout << "Пустая строка";
        return 0;
    }
    if (!valid_inf(infix)) {
        return 0;
    }
    cout << "Введённая строка проверкой : " << infix << endl;

    string postfixExpression = infixToPostfix(infix);
}

```

```

cout << "Постфиксная запись: " << postfixExpression << endl << endl;

int xmin, xmax, h;
cout << "Введите xmin: ";
cin >> xmin;
cout << "Введите xmax: ";
cin >> xmax;
cout << "Введите h: ";
cin >> h;
cout << endl;

for (int i = xmin; i <= xmax; i = i + h) {
    string dubler = postfixExpression;
    for (int j = 0; j < postfixExpression.size(); j++) {
        if (postfixExpression[j] == 'x') {
            dubler.replace(j, 1, to_string(i));
        }
    }
    if (evaluatePostfix(dubler) == std::numeric_limits<float>::infinity()) {
        cout << "x = " << i << "    f(x) = " << "---" <<
            endl;
    }
    else {
        cout << "x = " << i << "    f(x) = " <<
            evaluatePostfix(dubler) << endl;
    }
    if (i + h > xmax && i < xmax) {
        for (int j = 0; j < postfixExpression.size(); j
            ++){
            if (postfixExpression[j] == 'x') {
                dubler.replace(j, 1, to_string(
                    xmax));
            }
        }
        if (evaluatePostfix(dubler) == std::
            numeric_limits<float>::infinity()) {
            cout << "x = " << xmax << "    f(x) = " <<
                "---" << endl;
        }
        else {
            cout << "x = " << xmax << "    f(x) = " <<

```

```

                                evaluatePostfix(dubler) << endl;
                                }
                        }
    }

    return 0;
}

```

В программе данные считываются из файла, считанная строка проверяется на корректность написания. Далее, если данные записаны правильно, то строка из инфиксной записи переводится в постфксную. После того, как пользователь ввел границы аргумента и его шаг, программа создает таблицу с вычисленными значениями.

4 Исследовательская часть

Далее на рисунках представлен результат работы программы и входной файл с инффиксным выражением.

Результат работы программы, где во входном файле записана строка в инффиксной записи, она считывается и проверяется на валидность. Далее переводится в постфиксную запись и при введенных данных строится таблица. Результат приведён на рис. 1.

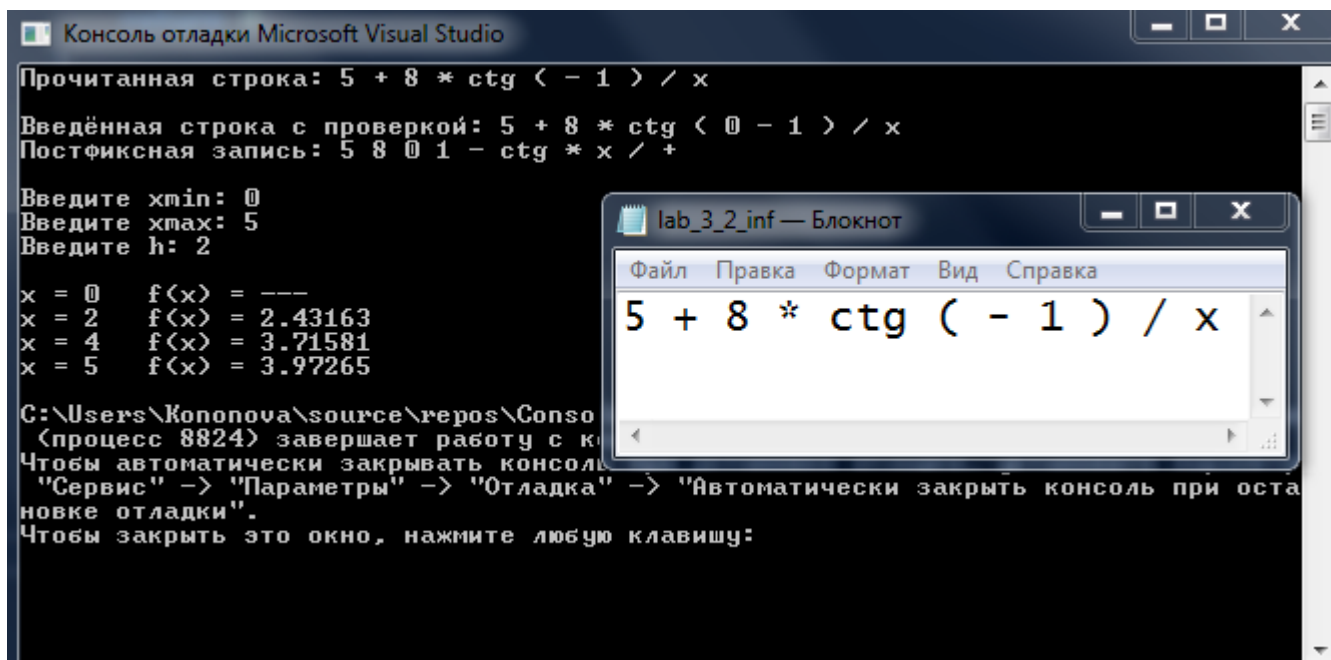
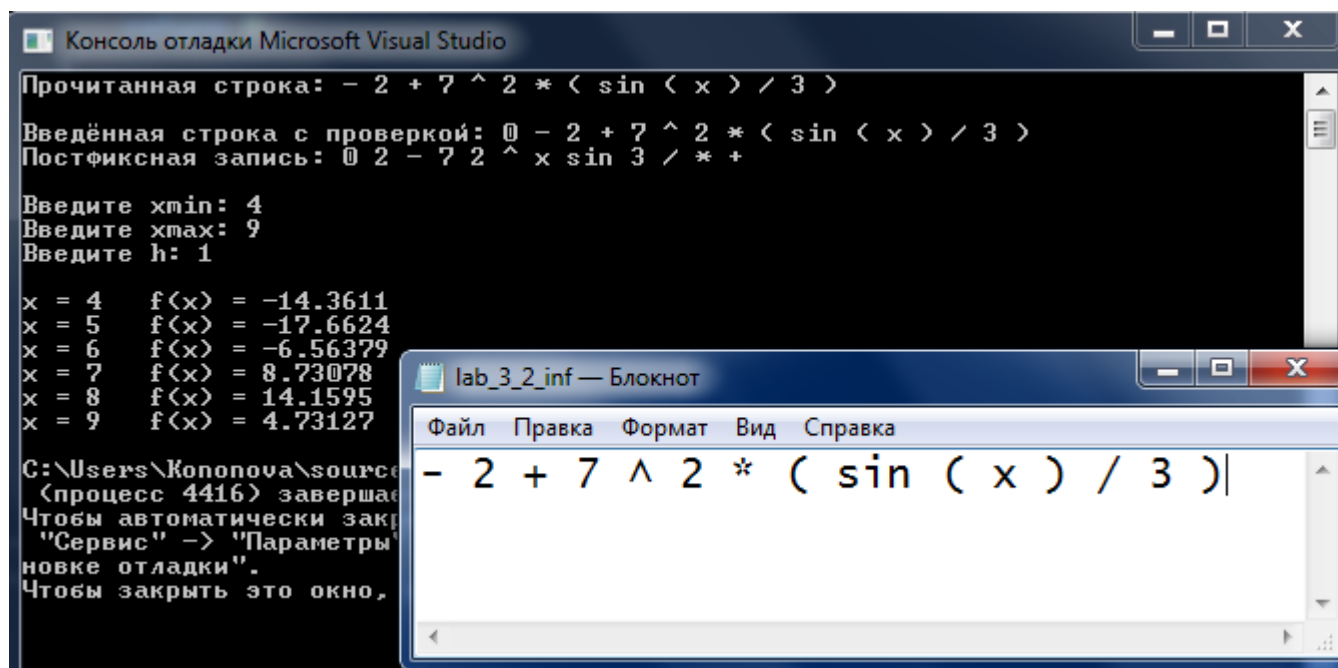


Рис. 1 – Пример №1

Результат работы программы, где во входном файле записана строка в инфиксной записи, она считывается и проверяется на валидность. Далее переводится в постфиксную запись и при введенных данных строится таблица. Результат приведён на рис. 2.



```
Консоль отладки Microsoft Visual Studio
Прочитанная строка: - 2 + 7 ^ 2 * < sin < x > / 3 >
Введённая строка с проверкой: @ - 2 + 7 ^ 2 * < sin < x > / 3 >
Постфиксная запись: @ 2 - 7 2 ^ x sin 3 / * +

Введите xmin: 4
Введите xmax: 9
Введите h: 1

x = 4    f(x) = -14.3611
x = 5    f(x) = -17.6624
x = 6    f(x) = -6.56379
x = 7    f(x) = 8.73078
x = 8    f(x) = 14.1595
x = 9    f(x) = 4.73127

C:\Users\Kononova\source
(процесс 4416) завершае
Чтобы автоматически закр
"Сервис" -> "Параметры
новке отладки".
Чтобы закрыть это окно,
```

```
lab_3_2_inf — Блокнот
Файл  Правка  Формат  Вид  Справка
- 2 + 7 ^ 2 * ( sin ( x ) / 3 )|
```

Рис. 2 – Пример №2

Результат работы программы, где во входном файле записана строка в инфиксной записи, она считывается и проверяется на валидность. Далее переводится в постфиксную запись и при введенных данных строится таблица. Результат приведён на рис. 3.

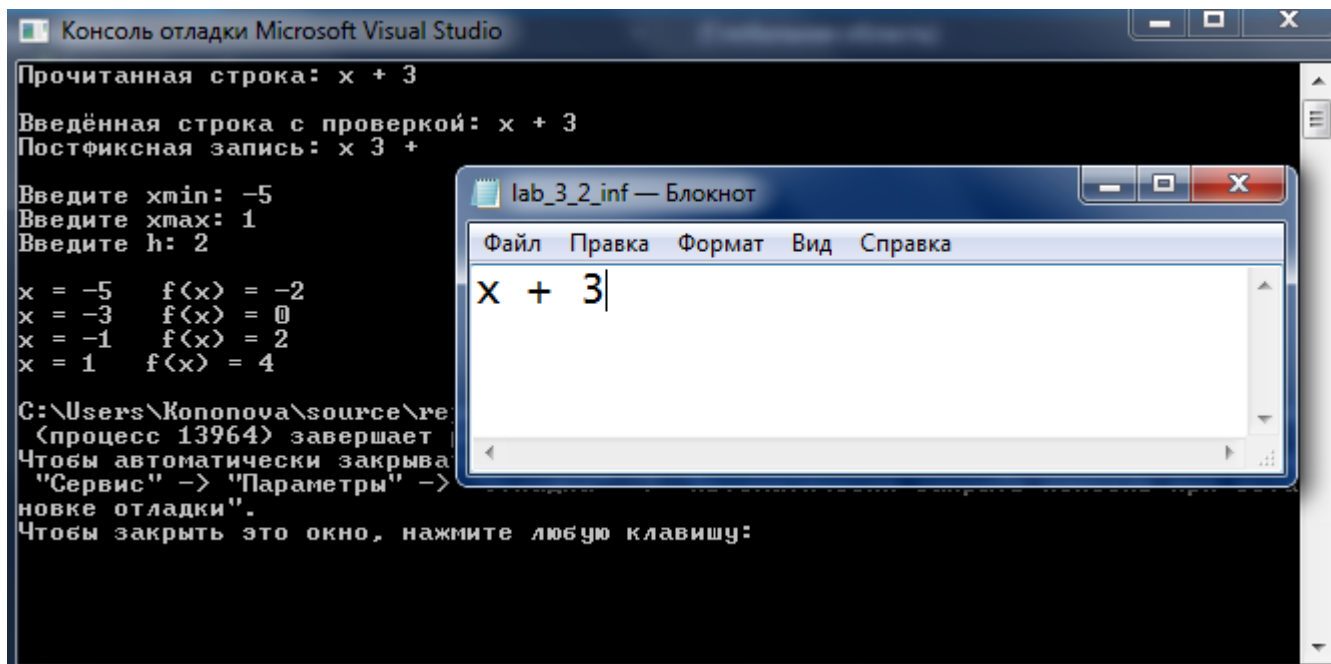


Рис. 3 – Пример №3

Результат работы программы при неверном количестве скобок приведён на рис. 4.

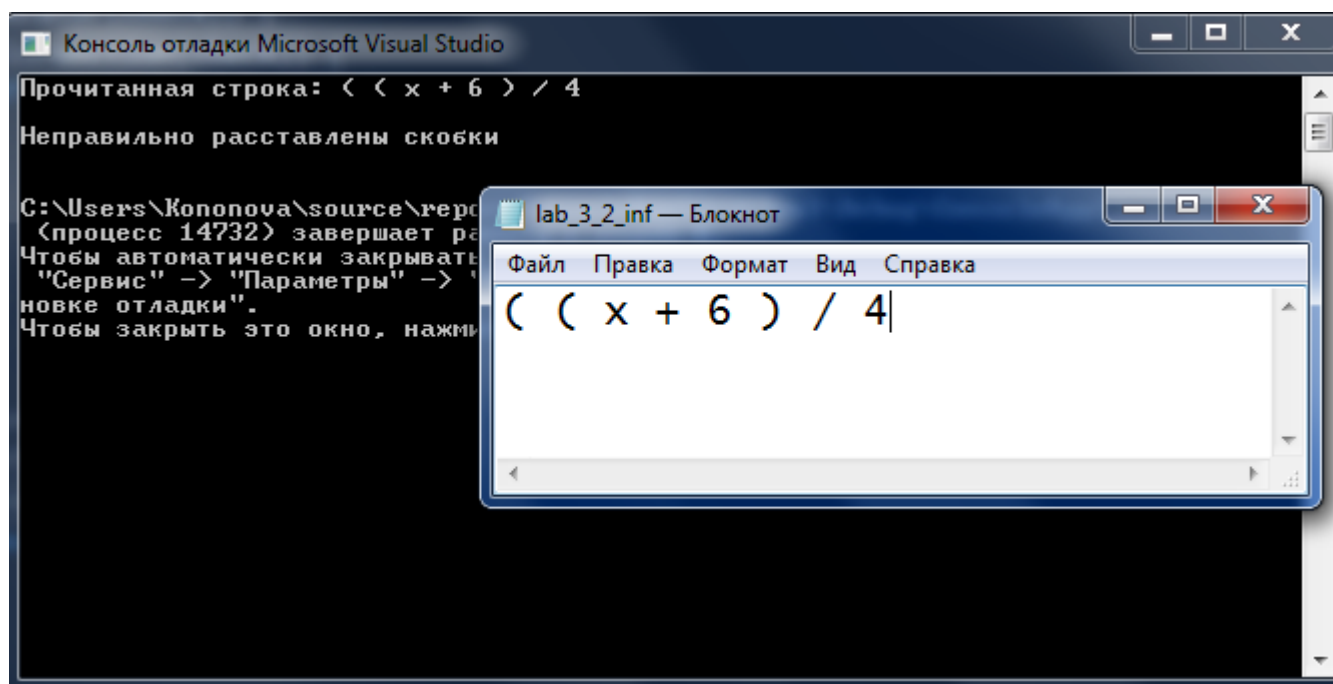


Рис. 4 – Пример №4

Результат работы программы при неверно введенных значениях выражения приведен на рис. 5.

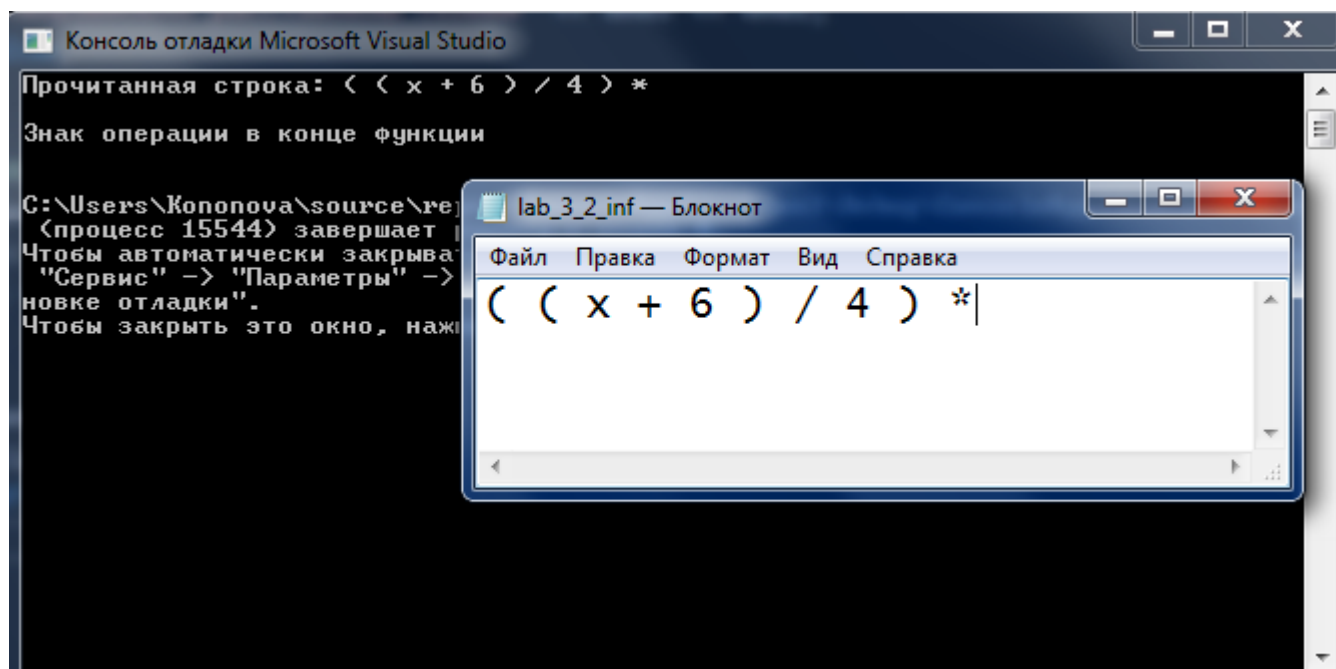


Рис. 5 – Пример №5

Результат работы программы при неверно введенных значениях выражения приведён на рис. 6.

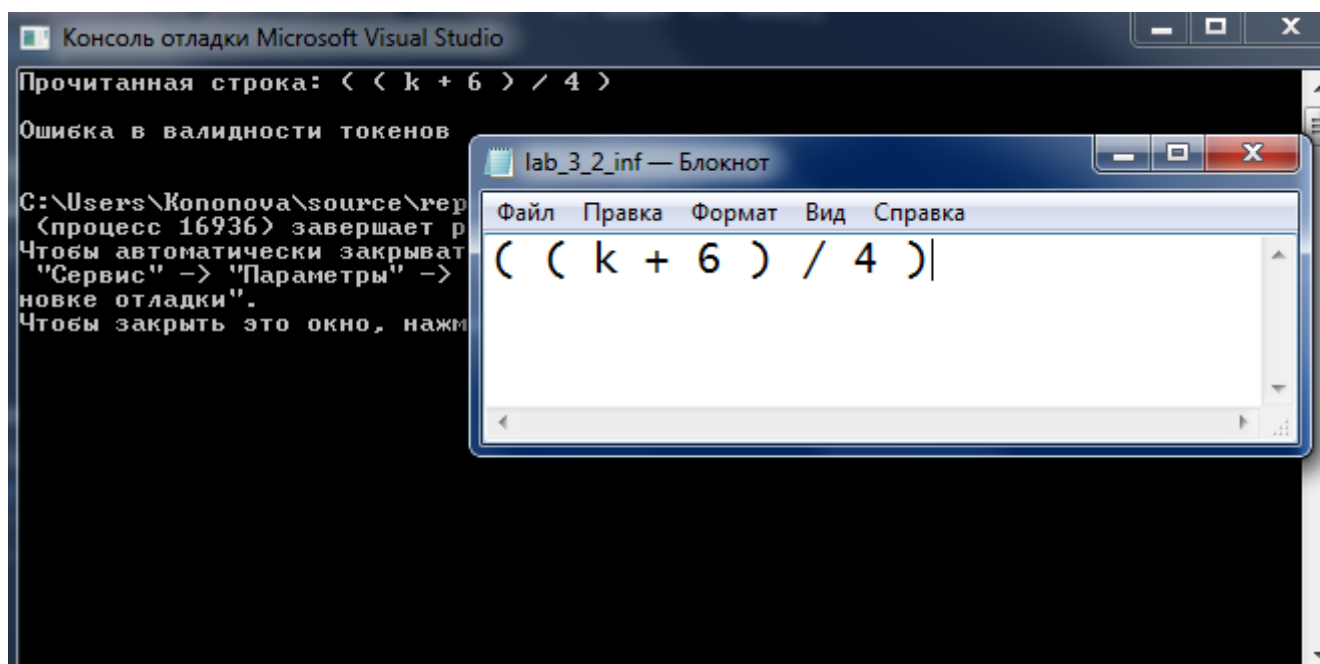
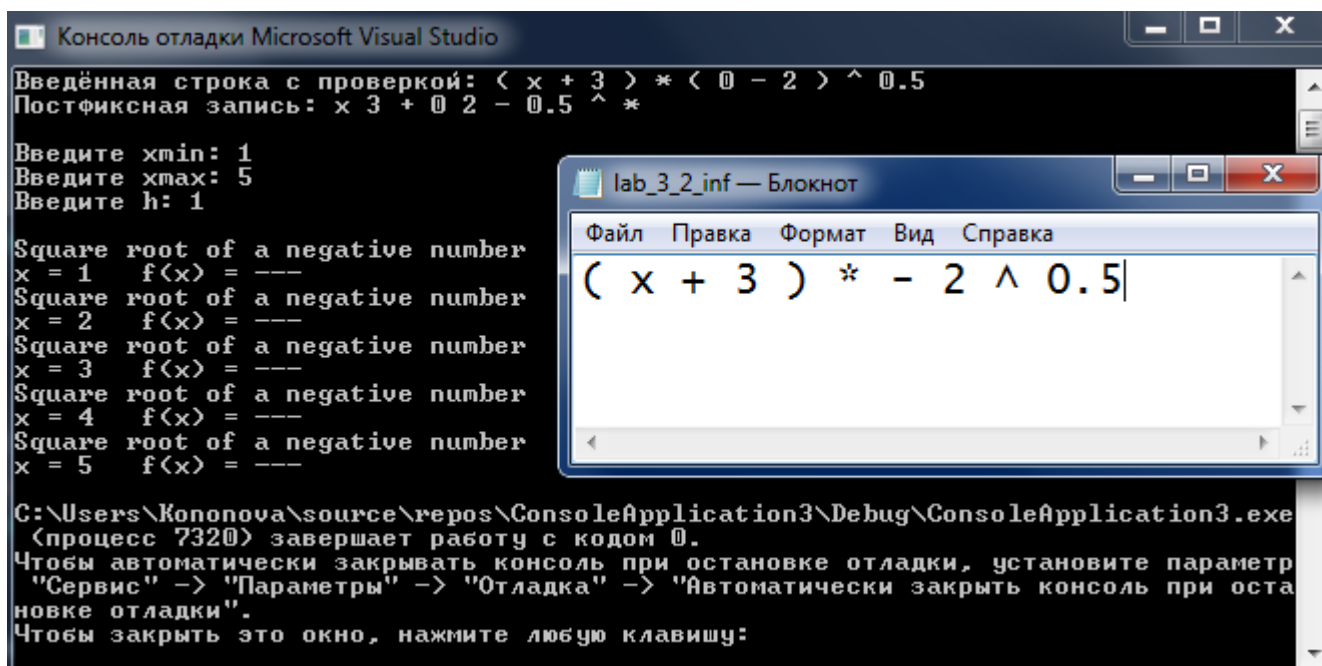


Рис. 6 – Пример №6

Результат работы программы при извлечении отрицательного числа из-под корня приведён на рис. 7.



The screenshot shows the Microsoft Visual Studio environment. The main console window displays the following text:

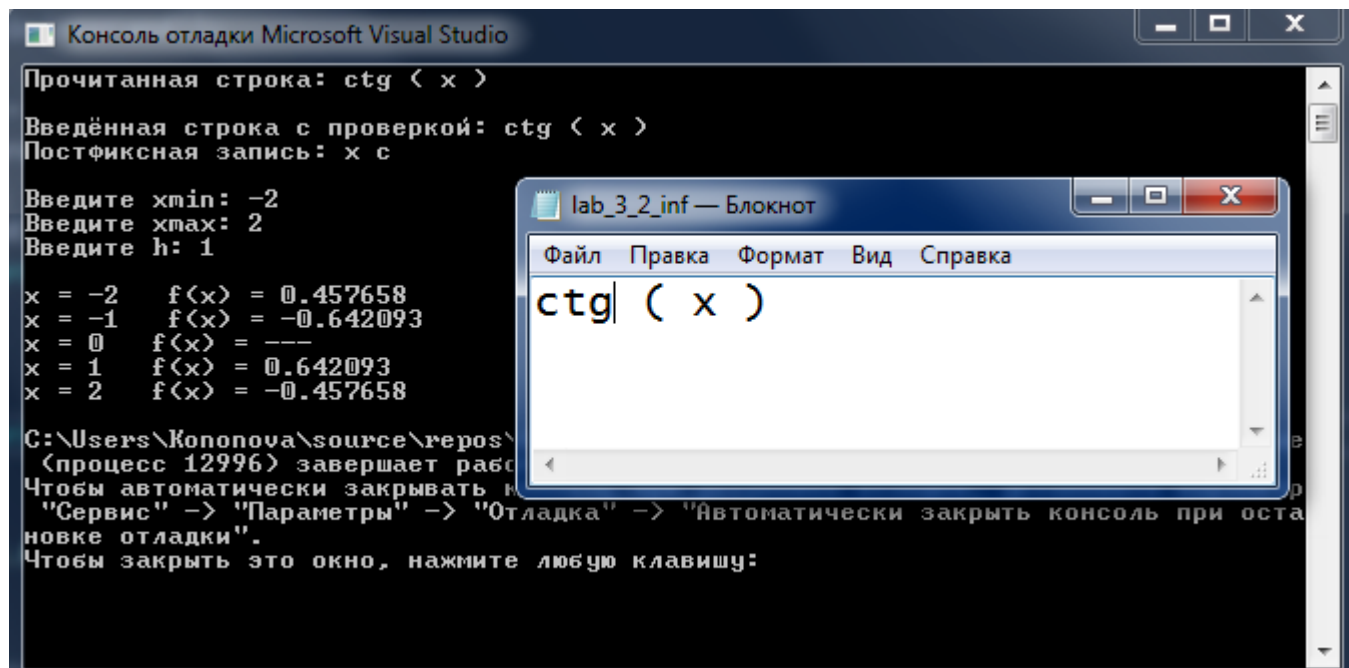
```
Консоль отладки Microsoft Visual Studio
Введённая строка с проверкой: < x + 3 > * < 0 - 2 > ^ 0.5
Постфиксная запись: x 3 + 0 2 - 0.5 ^ *
Введите xmin: 1
Введите xmax: 5
Введите h: 1
Square root of a negative number
x = 1    f(x) = ---
Square root of a negative number
x = 2    f(x) = ---
Square root of a negative number
x = 3    f(x) = ---
Square root of a negative number
x = 4    f(x) = ---
Square root of a negative number
x = 5    f(x) = ---
C:\Users\Kononova\source\repos\ConsoleApplication3\Debug\ConsoleApplication3.exe
(процесс 7320) завершает работу с кодом 0.
Чтобы автоматически закрывать консоль при остановке отладки, установите параметр
"Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при оста-
новке отладки".
Чтобы закрыть это окно, нажмите любую клавишу:
```

Overlaid on the console is a Notepad window titled "lab_3_2_inf — Блокнот". It contains the following text:

```
Файл  Правка  Формат  Вид  Справка
( x + 3 ) * - 2 ^ 0.5|
```

Рис. 7 – Пример №7

Результат работы программы при вычислении таблицы для котангенса приведён на рис. 8.



```
Консоль отладки Microsoft Visual Studio
Прочитанная строка: ctg < x >
Введённая строка с проверкой: ctg < x >
Постфиксная запись: x с
Введите xmin: -2
Введите xmax: 2
Введите h: 1
x = -2    f(x) = 0.457658
x = -1    f(x) = -0.642093
x = 0      f(x) = ---
x = 1      f(x) = 0.642093
x = 2      f(x) = -0.457658
C:\Users\Kononova\source\repos\
<процесс 12996> завершает работу
Чтобы автоматически закрывать консоль при оставлении отладки, перейдите в меню
"Сервис" -> "Параметры" -> "Отладка" -> "Автоматически закрыть консоль при оставлении отладки".
Чтобы закрыть это окно, нажмите любую клавишу:
```

lab_3_2_inf — Блокнот

Файл Правка Формат Вид Справка

ctg (x)

Рис. 8 – Пример №8

Результат работы программы при неправильно введенных операторах приведён на рис. 9.

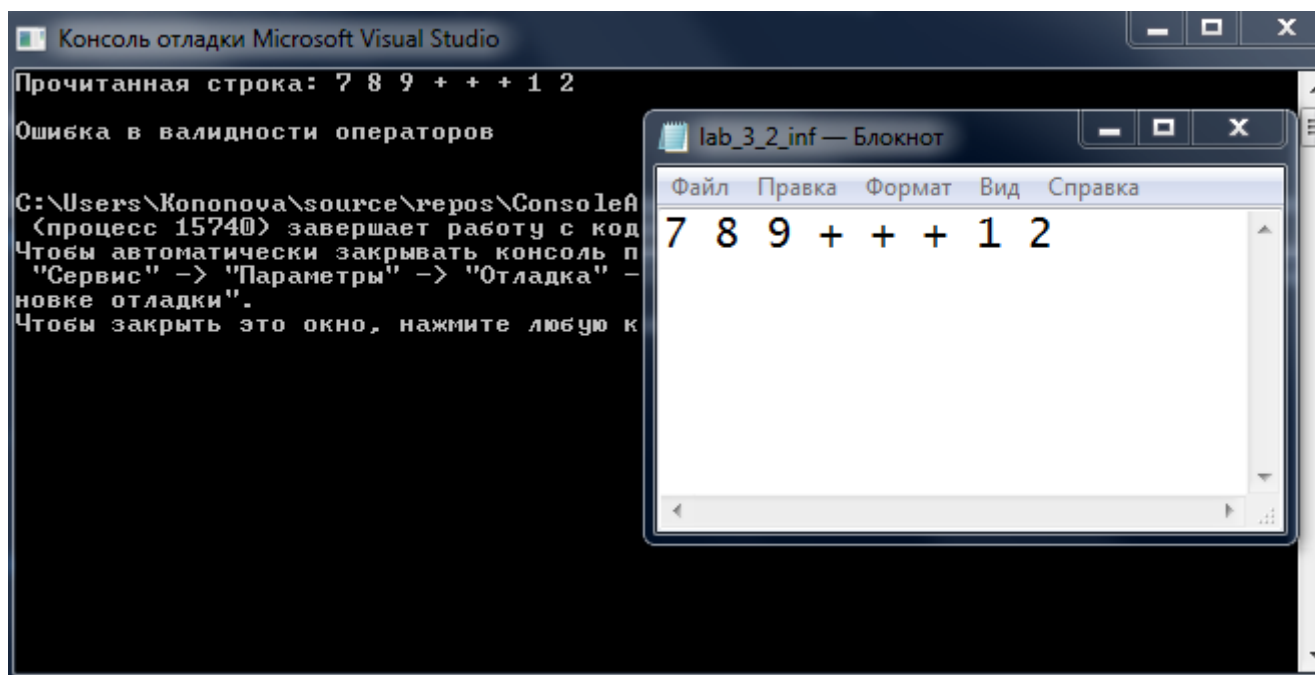


Рис. 9 – Пример №9

Заключение

Цель работы достигнута: разработана программа для вычисления значений функции с использованием обратной польской записи.

В результате выполнения лабораторной работы были выполнены все задачи.

1. Описаны две формы записи — инфиксная и постфиксная.
2. Разработан алгоритм проверки введенной инфиксной записи.
3. Реализован алгоритм перевода из инфиксной записи в постфиксную форму записи.
4. Составлена таблица значений функции, при заданных границах и шаге аргумента.
5. Выполнено тестирование реализации разработанных алгоритмов.

Список литературы

1. Е.А. Конова , Г.А. Поллак, А.М. Ткачев. Структуры данных. Программирование на языке С и С++. Учебное пособие, под ред. Поллак Г.А. — Челябинск: Издательство ЮУрГУ, 2004. — 106 с.